Real-Time Crowd Dynamics & Stampede Prevention Engine

Blueprint for the first working prototype: one monitored zone, one camera, one transit feed, a 20-minute density forecast with countermeasures.

Built on the existing **Headcounter** codebase (YOLO head detection, tracking, tripwire and ROI counting), extended with GTFS-Realtime ingestion and a Claude-based analysis layer.

Status: draft for build team review · Date: 3 November 2026 · Version 0.1

Decision-support prototype. Not a certified life-safety system; see Section 11.

Contents

- 1. Purpose and success criteria
- 2. Scope of Prototype 1
- 3. System architecture
- 4. Inputs and sensors
- 5. What already exists in the codebase
- 6. Component design
- 7. Data contracts
- 8. Site setup and calibration
- 9. Build plan (4 weeks)
- 10. Test and validation plan
- 11. Risks, limitations and safety boundaries
- 12. Operations, cost and hardware
- Appendix A. Configuration reference
- Appendix B. API reference
- Appendix C. Glossary

1. Purpose and success criteria

Prototype 1 answers one question: **can we turn a single CCTV feed plus upstream transit arrivals into a usable 20-minute crowd-density forecast and a concrete action list for an operator?** It is a decision-support tool for a human controller, not an automated actuator. Everything the model recommends is advisory until Section 10's validation gates are passed.

Success criteria

#	Criterion	Target	How measured
S1	Head-count accuracy in the monitored ROI	Mean absolute error <= 10% at densities up to 2 pax/m^2	Manual count on 30 sampled frames vs. pipeline count
S2	Flow-rate accuracy (inflow, outflow)	Within 20% of manual tally over 5-minute windows	Two observers with clickers at the tripwire
S3	Forecast calibration	T+10 density within +/-0.5 pax/m^2 of observed, 70% of windows	Replay logged telemetry; compare forecast vs. later observed density
S4	Risk-state usefulness	No missed CRITICAL in staged scenarios; <= 1 false CRITICAL per 8-hour run	Scenario runs (Section 10) + soak test
S5	Latency	Analysis endpoint returns in <= 60 s; dashboard refresh <= 2 s	Server logs
S6	Operator comprehension	Controller can state the risk level and first action within 15 s of reading the panel	Walkthrough with 3 operators

2. Scope of Prototype 1

In scope

- One zone: a corridor, platform access or gate area with a clear entry line, covered by one fixed camera.
- Live head detection, tracking, ROI density and bidirectional tripwire counting (existing Headcounter).
- One GTFS-Realtime source (TripUpdates, and VehiclePositions where published) mapped to the stop(s) that feed the zone.
- Telemetry derivation: density, inflow/outflow per minute, trend, upstream arrival pulses.
- Claude analysis producing a validated JSON assessment: 4 forecast horizons, LOS, surge probability, risk state, playbook, actuation flags, confidence, missing inputs.
- Dashboard panel showing the latest assessment; webhook alert on ADVISORY or above.
- Logging of every telemetry snapshot and assessment for offline validation.

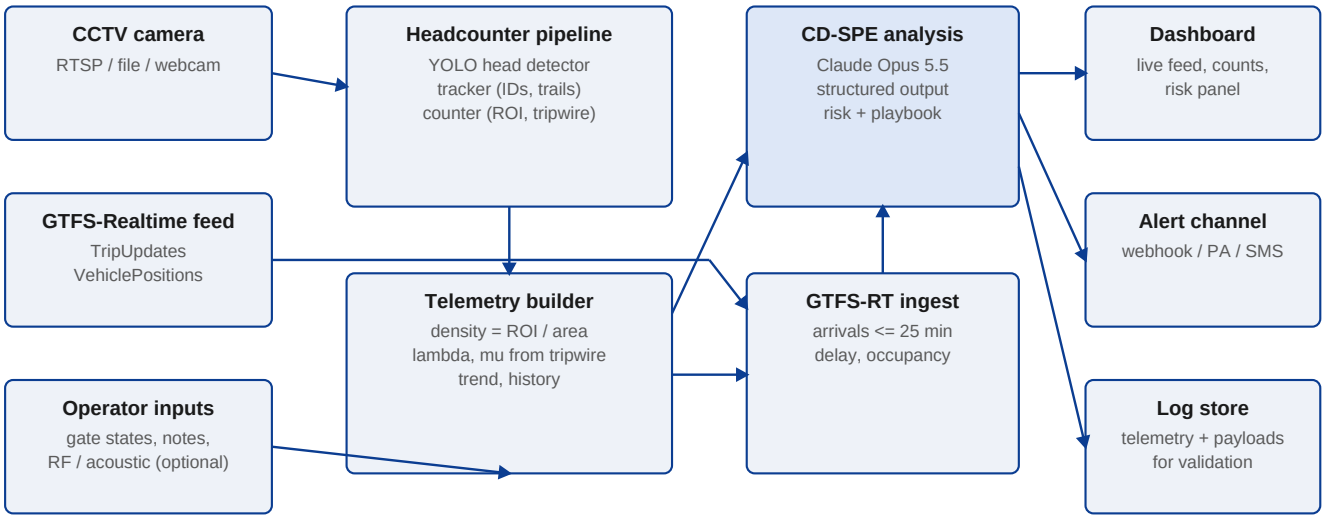
Out of scope (Prototype 2+)

- Multiple cameras or zones, camera-to-camera re-identification.
- Wi-Fi/BLE probe sensing, cellular handover data, acoustic panic detection. The schema already accepts these as optional inputs so they can be added without changing the model contract.
- Automatic actuation of gates, escalators, signage or PA. Prototype 1 emits flags only.
- Physics simulation of pedestrian movement (social-force or cellular automata). The forecast is model-reasoned from telemetry and flow conservation.
- Fare-tap or turnstile integration.

Guiding principle: make one zone work end to end and prove the forecast is calibrated before adding sensors. Every extra modality is a calibration problem; with one zone we can actually measure ground truth.

3. System architecture

Everything runs in one Python process (FastAPI + the Headcounter pipeline) on an edge machine next to the camera. The Claude call is the only external dependency at runtime; GTFS-RT is polled over HTTPS.



FastAPI app (app.py): /video_feed, /api/status, /api/analyze, /api/analyze/last

Runtime flow of one analysis cycle

- The video loop runs continuously: each frame goes through detector -> tracker -> counter, and the counter appends a history sample every 0.5 s.
- Every N seconds (default 120) a scheduler, or an operator pressing *Run analysis*, calls `POST /api/analyze`.
- `collect_telemetry()` reads the current metrics and the last 60 s of history and derives density, lambda, mu and trend.
- `fetch_gtfs_rt()` fetches TripUpdates and VehiclePositions, keeps arrivals at the zone's stops in the next 25 minutes, and attaches occupancy if present.
- The telemetry JSON is sent to Claude with the CD-SPE system prompt (cached) and a strict output schema. The response is validated into `CrowdAssessment`.
- The server builds the section-5 payload, stores the result as the last analysis, writes it to the log, and (if risk >= ADVISORY) posts to the alert webhook.
- The dashboard polls `GET /api/analyze/last` and renders the report and the risk badge.

4. Inputs and sensors

Input	Source	What it gives the model	Prototype 1 status
CCTV head counts	Headcounter pipeline (YOLO11n-head on MPS/CUDA)	ROI count -> density; tripwire IN/OUT -> lambda, mu; count trend	Live
GTFS-RT TripUpdates	Agency feed (protobuf)	Inbound arrivals with minutes-out and delay; simultaneous delayed arrivals = compound pulse	Live when URL + stop_ids configured
GTFS-RT VehiclePositions	Agency feed (protobuf)	occupancy_status / occupancy_percentage per arriving vehicle	Live if the agency publishes occupancy; else alighting assumption applies

Input	Source	What it gives the model	Prototype 1 status
Zone topology	config.py (ZONE_TOPOLOGY) or request body	Choke widths, gates open/closed, stairs, one-way rules	Static text, operator-maintained
Operator notes	Request body	Event milestones, closures, weather, anything the sensors cannot see	Manual
RF probes, acoustic, environment	extra_telemetry in request body	Reserved keys; model lists them under missing_inputs when null	Not connected

Delhi example: Open Transit Data (otd.delhi.gov.in) publishes static GTFS and realtime VehiclePositions for DTC/cluster buses with an API key. Occupancy fields are not published, so the alighting assumption (`GTFS_DEFAULT_ALIGHTING_PER_ARRIVAL`) carries the pulse estimate. Calibrate it on site: count alightings from 10 arrivals and set the mean.

5. What already exists in the codebase

The Headcounter repository is the starting point. The table shows each module, its role, and what Prototype 1 changes.

Module	Role today	Change for Prototype 1
config.py	Single source of settings: server, sources, models, thresholds, tripwire/ROI defaults, colours. Section 7 (analysis) added.	Fill ZONE_*, GTFS_* values for the site
engine/detector.py	Loads YOLO head model, auto-selects MPS/CUDA/CPU, downloads weights	None
engine/tracker.py	Greedy distance+IoU matcher, 15-frame grace, 30-point trails	Tune MAX_TRACKING_DISTANCE for camera height; see Section 8
engine/counter.py	Frame count, ROI count, peak, unique IDs, IN/OUT crossings, 0.5 s history (300 samples)	Raise max_history_len to 1200 (10 min) so flow windows are never starved
engine/pipeline.py	Opens source, runs detect/track/count, draws overlays, MJPEG generator	Add a persistent telemetry log writer
engine/crowd_analyst.py (new)	Telemetry builder, GTFS-RT ingest, CD-SPE prompt, structured-output schema, Claude call	Add scheduler + webhook (Section 6.6)
app.py	FastAPI: dashboard, /video_feed, status, history, source switching, upload, tripwire/ROI, CSV export, /api/analyze	Add /api/analyze/schedule, alert dispatch
templates/index.html	Single-page surveillance dashboard	Add analysis panel (Section 6.7)
cli.py, download_models.py	Offline processing and model fetch	None

6. Component design

6.1 Detection, tracking, counting (existing)

The pipeline is already fit for a first prototype. Two behaviours matter for forecasting and must be understood by whoever calibrates the site:

- **Tripwire direction.** A crossing is IN when the cross product of the line vector and the motion vector is positive. In practice: draw the line from left to right as seen on screen, and the side *above* the line on screen is OUT, below is IN. Verify with a walk-through before trusting lambda/mu.
- **Unique IDs inflate under occlusion.** When the tracker loses a head for more than 15 frames it issues a new ID. `total_unique_heads` therefore over-counts in dense scenes; the model is told to weight ROI count and tripwire rates, not unique IDs.

6.2 Telemetry derivation

Implemented in `collect_telemetry()`. Over the last `FLOW_WINDOW_SECONDS` (60 s) of counter history:

```
inflow_per_min   = (in_count[last]  - in_count[first])  * 60 / window_seconds     # lambda
outflow_per_min  = (out_count[last] - out_count[first]) * 60 / window_seconds     # mu
trend_per_min    = (roi_count[last] - roi_count[first]) * 60 / window_seconds
density_pax_m2   = roi_head_count / ZONE_AREA_M2
```

Flow conservation gives the model its baseline: dN/dt = lambda - mu. If the tripwire is disabled, lambda and mu are zero and the model is instructed to fall back on trend and to lower confidence. The last 24 history samples (12 s) are included so the model can see short-term acceleration.

6.3 GTFS-Realtime ingestion

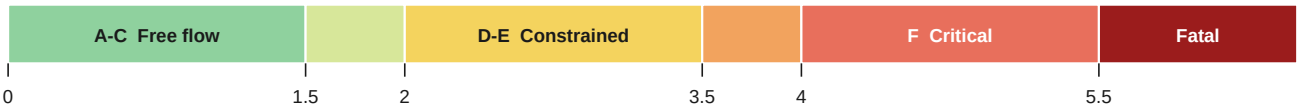
Implemented in `fetch_gtfs_rt()` with `gtfs-realtime-bindings`. For each configured stop_id it collects stop_time_updates whose arrival (or departure) falls between now-60 s and now+25 min, recording minutes-out, delay and schedule relationship. VehiclePositions are joined on trip_id to attach occupancy. Feed failures never block the analysis: the block reports `available: false` with a reason, and the model must list GTFS under missing inputs.

Mapping stops to the zone is a site task (Section 8). For a metro station, use the platform stop_ids whose exits lead into the monitored corridor. For a bus terminal, use the bay stop_ids.

6.4 Crowd physics reference

The system prompt encodes Fruin Level of Service so the model reasons against fixed thresholds. The critical threshold is a single configuration value (`DENSITY_CRITICAL = 4.0`) used both in the physics section and in time-to-critical, so narrative and actuation can never disagree.

Fruin Level of Service by density (persons per square metre)



Shockwave reasoning: above 4 pax/m^2 stop-and-go waves propagate backwards at roughly 1.0-1.5 m/s, so a stall at a 2.2 m turnstile bank reaches a queue 30 m back in about 20-30 s. The playbook's T+0-3 actions exist for exactly this window.

6.5 Claude analysis layer

Setting	Value	Why
Model	claude-opus-5-5	Strongest reasoning for multi-source fusion; thinking is always on
Effort	high (output_config.effort)	Forecasts need careful reasoning; medium is the model default and too shallow here
Output	output_config.format = json_schema from CrowdAssessment (Pydantic, extra=forbid)	Guaranteed parseable; no regex on markdown
max_tokens	16000, non-streaming	Report + JSON fit comfortably; stays under SDK timeout
System prompt	CD-SPE text with cache_control ephemeral	Stable prefix cached; only telemetry varies per call
Safety	betas server-side-fallback-2026-07-01, fallbacks = default	If the model declines, the API reruns on a fallback model in the same call
Error paths	refusal -> 502; max_tokens -> 502; no credential -> 503; rate limit -> 429	Operator sees why; scheduler backs off

Prompt rules that matter: the model must never invent readings for absent sensors, must list them in `missing_inputs`, and must set `confidence` accordingly. Actuation flags must be consistent with the playbook

text (both come from one structured response, so they cannot drift).

6.6 Scheduler and alerting (to build)

Add a background thread in `app.py` that calls `crowd_analyst.analyze()` every `ANALYSIS_INTERVAL_SECONDS` (120 default), skipping while paused or while a previous call is in flight. Adaptive cadence: 60 s when the last risk was ADVISORY or above, 300 s when NORMAL and density < 1 pax/m^2. On ADVISORY or above, POST the payload to `ALERT_WEBHOOK_URL` (Slack/Teams/SMS gateway). Persist each cycle as one JSON line in `logs/analysis.jsonl` and each telemetry snapshot in `logs/telemetry.jsonl`.

6.7 Dashboard panel (to build)

- Risk badge (NORMAL green, ADVISORY amber, CRITICAL red, SURGE_IMMINENT red pulsing) with time since last assessment.
- Forecast strip: four tiles for T+5/10/15/20 showing density, LOS and surge %.
- Playbook list in three time bands; confidence and missing-inputs line beneath.
- Upstream arrivals table from the telemetry block (minutes out, delay, occupancy).
- *Run analysis* button and an operator-notes field that posts to /api/analyze.
- Render `report_markdown` with a small client-side Markdown renderer (marked.js) for the full report view.

7. Data contracts

7.1 Telemetry sent to the model (abridged)

```
{
  "zone":       {"name": "...", "area_m2": 120.0, "topology": "..."},
  "thresholds": {"critical_pax_m2": 4.0, "fatal_pax_m2": 5.5},
  "cctv": {
    "captured_at_unix": 1762160000.0, "source": "rtsp://...", "is_paused": false,
    "detector": {"model": "yolo11n-head", "conf_threshold": 0.25, "fps": 31.2},
    "counts":   {"frame_head_count": 49, "roi_head_count": 47, "peak_head_count": 61,
                 "total_unique_heads": 412, "in_count_total": 117, "out_count_total": 39},
    "density":  {"area_m2": 120.0, "roi_density_pax_m2": 0.392},
    "flow":     {"window_seconds": 57.5, "inflow_per_min": 78.3, "outflow_per_min": 26.1,
                 "roi_count_trend_per_min": 26.1, "tripwire_enabled": true, "roi_enabled": true},
    "history_samples": [{"t_minus_s": 11.5, "count": 40, "roi": 38}, ...]
  },
  "gtfs_rt": {"available": true, "stop_ids": ["STOP_A"],
              "arrivals": [{"trip_id": "trip1", "route_id": "YL", "stop_id": "STOP_A",
                            "minutes_out": 7.0, "delay_s": 240, "schedule_relationship": "SCHEDULED"}],
              "vehicles": [{"trip_id": "trip1", "occupancy_status": "STANDING_ROOM_ONLY",
                            "occupancy_percentage": 85, "current_status": "IN_TRANSIT_TO"}],
              "assumed_alighting_per_arrival": 150},
  "rf_probes": null, "acoustic": null, "environment": null,
  "operator_notes": "Platform 2 train held 6 min at outer signal"
}
```

7.2 Assessment returned by the model (CrowdAssessment schema)

Field	Type	Meaning
synthesis.active_bottlenecks	string[]	Named choke points currently binding
synthesis.inflow_per_min / outflow_per_min	number	Model's reconciled lambda and mu
synthesis.sensor_concordance	string	Agreement or conflict between CCTV and GTFS
trajectory[]	{horizon_min, density_pax_m2, flow_velocity_ms, los A-F, surge_probability_pct}	Four rows: 5, 10, 15, 20
threat.risk_level	NORMAL | ADVISORY | CRITICAL | SURGE_IMMINENT	Drives badge, alerting and cadence

Field	Type	Meaning
threat.primary_failure_mode	string	e.g. uni-directional crush, counter-flow shear, dead-end bottleneck
threat.minutes_to_critical	number | null	Time until density >= DENSITY_CRITICAL, null if not within 20 min
playbook.t0_to_3_min / t3_to_10_min / t10_to_20_min	string[]	Ordered actions per time band
actuation	7 booleans	trigger_pa_announcement, update_digital_signage, halt_inbound_escalators, meter_turnstiles, divert_to_bypass, hold_upstream_trains, close_platform_stairs
confidence	0-1	Lower when inputs are missing or conflicting
missing_inputs / assumptions	string[]	Audit trail for the operator
markdown_report	string	Sections 1-4 of the operator report

7.3 Actuation payload (section 5, built by the server)

```
{
  "generated_at_unix": 1762160061.2, "zone": "Demo corridor (camera 1)",
  "model": "claude-opus-5-5", "risk_level": "ADVISORY",
  "primary_failure_mode": "Uni-directional crush at turnstile bank",
  "minutes_to_critical": 14.0, "current_density_pax_m2": 0.392,
  "inflow_per_min": 78.3, "outflow_per_min": 26.1,
  "forecast": [
    {"horizon_min": 5, "density_pax_m2": 2.1, "flow_velocity_ms": 0.9, "los": "D", "surge_probability_pct": 15},
    ...
  ],
  "actuation": {
    "trigger_pa_announcement": true, "update_digital_signage": true, "halt_inbound_escalators": false,
    "meter_turnstiles": true, "divert_to_bypass": false, "hold_upstream_trains": false,
    "close_platform_stairs": false
  },
  "confidence": 0.55,
  "missing_inputs": ["rf_probes", "acoustic", "environment", "vehicle occupancy"]
}
```

8. Site setup and calibration

This is half the work of the prototype and it is physical, not software. Allow two full days on site.

Step	Task	Output / where it goes
8.1	Choose the zone: one corridor or gate line with a single dominant flow direction and a camera position 3-5 m high looking along the flow at 30-45 degrees down. Avoid backlight.	Photo, sketch, camera mount
8.2	Measure the usable floor area the ROI will cover (laser measure, subtract pillars, shoe stands, barriers).	ZONE_AREA_M2
8.3	Draw the ROI polygon on the dashboard to match the measured area. Keep it to floor the camera sees fully; crop out far background where heads are < 12 px.	/api/roi (pixel points)
8.4	Draw the tripwire across the entry; walk through 10 times each direction and confirm IN/OUT. Swap endpoints if reversed.	/api/tripwire
8.5	Tune detection: confidence 0.20-0.35; check recall on 20 frames at the densest time of day. Tune MAX_TRACKING_DISTANCE (px per frame) to the walking speed at this camera height.	config.py
8.6	Write ZONE_TOPOLOGY: widths in metres, number of gates and which are open, stairs/escalators and direction, one-way rules, dead ends.	config.py

Step	Task	Output / where it goes
8.7	Map GTFS stop_ids: from the static GTFS stops.txt, pick the stops whose alighting passengers enter this zone. Confirm the realtime feed carries those stop_ids.	GTFS_STOP_IDS, feed URLs in env
8.8	Calibrate alighting: count passengers alighting from 10 arrivals; set the mean.	GTFS_DEFAULT_ALIGHTING_PER_ARRIVAL
8.9	Ground-truth session: two observers count tripwire crossings with clickers for 3 x 5-minute windows; record manual ROI counts at 30 timestamps.	Validation sheet (Section 10)

9. Build plan (4 weeks)

One engineer full-time plus a part-time site contact. Weeks are sequential; each ends with a demo.

Week	Deliverables	Exit demo
1 Foundations	Repo tidy (remove stale reports), history length to 1200, telemetry + analysis JSONL logging, scheduler thread with adaptive cadence, webhook alert, unit tests for collect_telemetry and fetch_gtfs_rt (synthetic protobuf).	Analysis runs every 2 min on the demo clip; log files fill; alert fires on a forced ADVISORY
2 Dashboard + GTFS	Analysis panel in index.html (badge, forecast strip, playbook, arrivals table, run button, notes field). Connect the real GTFS-RT feed; stop mapping; alighting assumption.	Live feed + panel + real upstream arrivals shown for the chosen station
3 Site install	Camera mounted, ROI/tripwire drawn, area measured, topology written, detection tuned, ground-truth session (Section 8).	Prototype running on site for one full peak period with logs
4 Validation + hardening	Replay evaluation of forecasts vs. observed (S3), staged scenarios (Section 10), false-alert review, operator walkthrough (S6), fix list, cost report, v0.1 tag.	Validation report; go/no-go for Prototype 2

Definition of done for Prototype 1: all six success criteria measured and reported, even if some are missed. A missed target with data is a result; an unmeasured target is not.

10. Test and validation plan

10.1 Unit and integration tests

- `collect_telemetry`: rising queue, empty history, paused pipeline, tripwire disabled (lambda = mu = 0).
- `fetch_gtfs_rt`: synthetic FeedMessage with in-window, out-of-window and other-stop arrivals; occupancy join; feed timeout; bad key (HTTP 401) -> available false.
- Schema: every model response validates against `CrowdAssessment`; four trajectory rows with horizons 5/10/15/20.
- Endpoint: 503 without credentials, 429 mapping, 404 on /last before first run.

10.2 Staged scenarios (replayed from recorded clips or live with marshals)

Scenario	Setup	Expected
A. Steady free flow	Normal off-peak clip, no upstream arrivals	NORMAL, LOS A-C at all horizons, all actuation flags false, confidence reflects missing RF/acoustic
B. Single arrival pulse	One GTFS arrival in 4 min, density 0.8	Density rises at T+5 then recovers; NORMAL or ADVISORY; PA/signage only
C. Compound pulse	Two delayed arrivals within 90 s, current density 1.8, mu limited by 2 open gates	ADVISORY or CRITICAL; meter_turnstiles true; hold_upstream_trains considered at T+10-20; minutes_to_critical populated

Scenario	Setup	Expected
D. Outflow collapse	Operator note: downstream stairs closed; lambda > 0, mu ~ 0	CRITICAL/SURGE_IMMINENT; dead-end bottleneck named; halt escalators, hold trains
E. Sensor conflict	GTFS shows large pulse but CCTV count flat (camera occluded)	Model flags concordance anomaly, lowers confidence, does not fabricate density
F. Feed outage	GTFS URL unreachable	Analysis still returns; gtfs listed in missing_inputs; confidence reduced

10.3 Forecast calibration (S3)

From the JSONL logs, for every assessment at time t take the observed ROI density at t+5, t+10, t+15, t+20 and compute absolute error per horizon. Report MAE and the share of windows inside +/-0.5 pax/m^2. Also report Brier score for surge_probability against whether density crossed 3.5 in the horizon. Expect T+5 to be good and T+20 to be weak; the purpose is to know by how much.

10.4 Soak test

Run 8 hours including a peak. Count ADVISORY+ alerts, review each against footage, classify true/false. Record API failures, latency distribution and token usage.

11. Risks, limitations and safety boundaries

Risk / limitation	Impact	Mitigation in Prototype 1
Not a certified life-safety system	Over-reliance could delay human judgement	Advisory only; no actuation; banner on dashboard; operator training
Transit-riders-only view	Walk-up crowds invisible upstream	Operator notes field; RF probes in Prototype 2
Tracker ID swaps in dense crowds	Unique counts inflate; crossings missed or doubled	Model weights ROI density and rates; validate S1/S2; consider ByteTrack in Prototype 2
Camera occlusion, low light, rain on lens	Density under-read while risk rises	Scenario E; GTFS acts as independent signal; confidence drops
Pixel-space ROI vs. real area	Density wrong if area mis-measured	Section 8.2-8.3 procedure; re-measure after any camera move
Single shared pipeline	Two dashboard tabs halve the frame rate	Use /api/frame polling for secondary viewers
LLM latency (10-60 s)	Forecast is stale by up to a minute	T+5 horizon absorbs it; cadence tightens under ADVISORY
Model refusal or API outage	No assessment	Server-side fallback; last-good payload shown with age; alert on 3 consecutive failures
Hallucinated sensor readings	False confidence	Prompt forbids it; schema forces missing_inputs; scenario E
Privacy	CCTV of the public	No faces stored; only counts and annotated frames in memory; logs hold numbers only; follow site CCTV policy

12. Operations, cost and hardware

12.1 Hardware

Item	Spec	Notes
Edge compute	Apple Mac mini (M-series) or NVIDIA Jetson Orin NX / small PC with RTX	Current code runs ~30-40 FPS with YOLO11n-head on Apple MPS

Item	Spec	Notes
Camera	IP camera, RTSP, >= 1080p, 25 fps, wide dynamic range	Fixed mount; PoE preferred
Network	Wired to camera; outbound HTTPS to api.anthropic.com and the GTFS feed	Allow-list both; no inbound needed
Display	Any browser on the control-room LAN	Dashboard on port 8000

12.2 Running

```
export ANTHROPIC_API_KEY=sk-ant-...
export GTFS_RT_TRIP_UPDATES_URL="https://.../TripUpdates.pb?key=..."
export GTFS_RT_VEHICLE_POSITIONS_URL="https://.../VehiclePositions.pb?key=..."
./run_dashboard.sh                    # creates venv, installs requirements, starts uvicorn on :8000

curl -X POST localhost:8000/api/analyze -H 'content-type: application/json' \
     -d '{"area_m2": 120, "operator_notes": "Platform 2 train held 6 min"}'
```

12.3 Cost estimate

Per analysis call on claude-opus-5-5 (USD 4 per million input tokens, USD 20 per million output): system prompt ~1,200 tokens (cached after the first call), telemetry ~1,500-2,500 tokens, output ~2,000-3,000 tokens including the report. Roughly **USD 0.05-0.08 per call**.

Cadence	Calls / hour	Approx. cost / hour	Cost / 12-hour day
Every 5 min (quiet)	12	USD 0.6-1.0	USD 7-12
Every 2 min (default)	30	USD 1.5-2.4	USD 18-29
Every 60 s (ADVISORY+)	60	USD 3-5	USD 36-58 (only during incidents)

The adaptive cadence in Section 6.6 keeps a typical day near the default row. Verify with `usage` in each response; `cache_read_input_tokens` should be non-zero from the second call onward.

Appendix A. Configuration reference (config.py, section 7)

Key	Default	Purpose
ANALYSIS_MODEL	claude-opus-5-5	Model for the assessment
ANALYSIS_EFFORT	high	Reasoning depth: low | medium | high | xhigh | max
ZONE_NAME	Demo corridor (camera 1)	Label in reports and payloads
ZONE_AREA_M2	120.0	Usable floor area under the ROI polygon
ZONE_TOPOLOGY	text	Widths, gates, stairs, one-way rules
DENSITY_CRITICAL / DENSITY_FATAL	4.0 / 5.5	LOS F threshold and fatal zone, pax/m^2
FLOW_WINDOW_SECONDS	60	Window for lambda, mu and trend
GTFS_RT_TRIP_UPDATES_URL	env var	Protobuf TripUpdates feed
GTFS_RT_VEHICLE_POSITIONS_URL	env var	Protobuf VehiclePositions feed
GTFS_STOP_IDS	[]	Stops feeding the zone
GTFS_LOOKAHEAD_MINUTES	25	Arrival horizon
GTFS_DEFAULT_ALIGHTING_PER_ARRIVAL	150	Pulse size when occupancy is absent
ANALYSIS_INTERVAL_SECONDS (to add)	120	Scheduler cadence
ALERT_WEBHOOK_URL (to add)	env var	POST target for ADVISORY+

Appendix B. API reference

Method / path	Body	Returns
GET /	-	Dashboard
GET /video_feed	-	MJPEG stream of annotated frames
GET /api/frame	-	Latest annotated JPEG
GET /api/status	-	Counts, FPS, model, thresholds
GET /api/history	-	Counter history samples
POST /api/set_source	{source_type, demo_id | url | index | filepath}	Switch camera / clip
POST /api/settings	{model_name, conf_threshold, is_paused, overlay_type, enabled}	Runtime settings
POST /api/tripwire	{enabled, pt1, pt2}	Set counting line
POST /api/roi	{enabled, points}	Set ROI polygon
POST /api/reset_counts	-	Reset counters
GET /api/export_csv	-	History as CSV
POST /api/analyze	{zone_name?, area_m2?, topology?, operator_notes?, extra_telemetry?}	telemetry, assessment, report_markdown, payload, usage
GET /api/analyze/last	-	Most recent analysis (404 if none)

Appendix C. Glossary

Term	Meaning
Lambda / mu	Inflow and outflow rates in persons per minute; dN/dt = lambda - mu
LOS	Fruin Level of Service, A (free) to F (jammed), by density
ROI	Region of interest: the floor polygon in which heads are counted for density
Tripwire	Virtual line; trajectories crossing it increment IN or OUT
Compound pulse	Two or more vehicle arrivals close together, delivering alighting passengers at once
GTFS / GTFS-RT	General Transit Feed Specification (static schedule) and its realtime protobuf feeds
Surge probability	Model's estimate that density exceeds the constrained band within the horizon
Structured output	API feature that constrains the model response to a JSON schema